

AI/ML Team Document

Context Compression Engine — Prompt Engineering & Compression Logic Guide

Your role: This is a 4th team worth splitting out from Backend, because the quality of your prompts is what makes or breaks the 70% compression / 95% accuracy targets — it's a distinct skill from API plumbing. You own the actual "intelligence" of the compression: chunk scoring, chunk summarization, and the accuracy-judging prompt. If your team is small, this role can be one person embedded inside Backend — but the tasks below should still be treated as a distinct phase of work.

1. Your Responsibilities

- Design and test the **chunk scoring prompt** (which parts of the context matter most)
- Design and test the **chunk compression/summarization prompt** (compress without losing meaning)
- Design and test the **question generation prompt** (auto-generate a fair test question when user doesn't provide one)
- Design and test the **similarity judging prompt** (score how close two answers are)
- Tune thresholds (score cutoffs, chunk size) based on real test runs
- Hand off finalized prompts + expected input/output formats to Backend for integration

2. Step-by-Step Build Plan

Step 1: Understand the Content Types You'll Handle (15 min)

Prepare 3 test samples before writing any prompts:

1. A code file (~150–300 lines) with some repeated imports/boilerplate
2. A log file (~200+ lines) with repeated timestamp/header patterns
3. A long chat/support transcript (~100+ messages) with filler ("thanks", "ok", "got it")

You'll use these throughout to test every prompt below.

Step 2: Chunk Scoring Prompt (1 hr)

Goal: given a batch of chunks, return a relevance/information-density score 1–10 for each, so low-value chunks can be compressed and high-value chunks preserved verbatim.

Draft prompt structure:

```
You are scoring chunks of text for information density and importance for future

For each chunk below, assign a score from 1 (pure filler/boilerplate, safe to remove) to 10 (critical unique information – specific facts, numbers, logic, named entities)

Return ONLY valid JSON, no other text, in this exact format:
[{"id": "chunk_1", "score": 7, "reason": "contains function logic and variable names"}]

Chunks:
{chunk_1_id}: {chunk_1_text}
{chunk_2_id}: {chunk_2_text}
...
```

Test this on all 3 sample types. Things to check:

- Does it correctly score repeated boilerplate low and unique logic/facts high?
- Does the JSON parse reliably? If Claude adds preamble text despite instructions, add: "Do not include any text before or after the JSON array."
- What's a good chunk batch size per call? Start with ~10 chunks per call, adjust based on response quality and latency.

Deliverable: finalized prompt template + documented expected JSON schema, handed to Backend.

Step 3: Chunk Compression Prompt (1 hr)

Goal: for chunks scored below the threshold (e.g. <6), compress them while preserving all critical details.

Draft prompt structure:

```
Compress the following text as much as possible while strictly preserving:
- All named entities (people, systems, variables, function names)
- All numbers, dates, and quantities
- All logical/causal relationships (if X then Y)
```

Remove filler words, redundant phrasing, and repeated explanations. Do not add ne
Return only the compressed text, no preamble.

Text:
{chunk_text}

Test this on all 3 sample types. Things to check:

- Are numbers/entities preserved exactly (not paraphrased or rounded)?
- Is the compression aggressive enough to hit the 70% target when combined with Stage 1?
- For code specifically: test whether Claude should compress code chunks at all, or just comments/docstrings, since compressing actual code syntax risks breaking correctness. **Recommendation: for code chunks, only compress comments/docstrings, never the code logic itself.** Add this as a conditional rule in the prompt or route code chunks through a different, more conservative prompt.

Deliverable: finalized prompt template, plus the code-specific conservative variant, handed to Backend.

Step 4: Question Generation Prompt (30 min)

Goal: when the user doesn't supply a test question, auto-generate one that fairly tests whether compression preserved the important information.

Draft prompt structure:

Given the following content, generate ONE specific question that:

- Can only be answered correctly by someone who read this content
- Tests a specific fact, number, or piece of logic (not a vague/general question)
- Has a clear, checkable answer

Return only the question, no preamble.

Content:
{original_text}

Test this on all 3 sample types and manually verify the generated questions are actually answerable and specific (not something generic like "What is this document about?").

Deliverable: finalized prompt, handed to Backend.

Step 5: Similarity Judging Prompt (45 min)

Goal: given two answers (one from full context, one from compressed context), return a 0–100 similarity/retention score.

Draft prompt structure:

```
Compare these two answers to the same question and rate how semantically similar/
on a scale of 0-100, where 100 means they convey identical factual content (wordi
and 0 means they are contradictory or unrelated.
```

```
Question: {question}
Answer A: {answer_full}
Answer B: {answer_compressed}
```

```
Return ONLY valid JSON in this format: {"score": 87, "reasoning": "brief explanat
```

Test this deliberately with:

- Two nearly identical answers → should score 90+
- Two answers with a clear factual contradiction → should score low (<40)
- This calibration matters — run at least 5 test pairs and sanity check the scores make sense before handing off

Deliverable: finalized prompt, handed to Backend.

Step 6: Threshold Tuning (1 hr, do this after Steps 2–5 are integrated by Backend)

Once Backend has the pipeline running end-to-end, run all 3 sample files through it and tune:

1. **Score threshold for compression** (currently assumed <6) — adjust up/down based on whether real compression ratio hits 70%+
2. **Chunk size** — if compression ratio is too low, try smaller chunks (more granular scoring); if quality drops, try larger chunks (more context per scoring decision)
3. **Batch size for scoring calls** — balance between fewer API calls (cheaper/faster) and scoring quality (too many chunks per call may degrade attention)

Document your final chosen values and *why* — this is useful content for the pitch deck (“we tuned X and found Y”).

Step 7: Fallback Behavior Definition (30 min)

Work with Backend to define what happens when:

- JSON parsing fails on the scoring prompt → fallback: treat all chunks as score 5 (medium), apply light compression uniformly
- A chunk compression call times out → fallback: keep that chunk verbatim (better to under-compress than to fail the whole run)

Document these fallback rules clearly so Backend can implement them without needing to re-check with you mid-build.

3. What “Done” Looks Like

- Four finalized, tested prompt templates (scoring, compression, question generation, similarity judging)
- Documented expected input/output JSON schema for each
- Documented threshold values (score cutoff, chunk size, batch size) with brief reasoning
- Documented fallback behavior for failure cases
- All four prompts have been manually tested against all 3 sample content types (code, logs, chat transcript)

4. Coordination Notes

- Hand off each prompt to Backend as soon as it's tested and finalized — don't wait until all four are done, since Backend needs to start wiring the pipeline in parallel
- Keep a shared doc/note of example inputs → outputs for each prompt, so Backend can write good test fixtures
- Flag early if any prompt seems unreliable (inconsistent JSON, inconsistent scoring) — better to redesign at hour 6 than discover it at hour 20